UnicoTrader E-Commerce Complete Build Roadmap

Phase-wise Roadmap and Task Specification
Customer Portal + Admin Portal + Backend API + Payments + Deployment

Project: unicotrader.com (Unico Traders, Surat) | Stack: Next.js + Node.js + PostgreSQL | November 2026

About this roadmap

Goal: take UnicoTrader from an enquiry-only WordPress catalogue to a complete, secure online store with a combined online + offline admin dashboard. This roadmap tells you exactly where to start, what to build next, and when a phase is truly finished - from the first Git commit to DNS cutover and post-launch growth. It follows the same style as the BookMyShow-lite roadmap: every task has a description, endpoints, rules, database changes and a 'Done When' check.

How to use it

- Work phase by phase. Move on only when every 'Done When' check of the phase passes.
- Write the code yourself. Read docs to understand concepts; do not copy-paste solutions.
- Make it work first, then make it clean, then make it fast.
- After each phase: add a git tag, update the README and write 5 lines on what you learned.
- Write tests along with each phase instead of leaving everything for Phase 13.
- Phase 0.2 and 0.9 are on the critical path: business answers and slow external approvals (KYC, DLT, WhatsApp) need waiting time, so start them in week 1.
- Parallel work is fine: once Phase 4 APIs have a contract, the customer portal (Phase 8) can start against mocked data.

Recommended stack (client confirmed: custom, not WooCommerce)

Layer	Technology
Customer portal	Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, Zustand
Admin portal	Next.js 14 (separate app), same UI kit, TanStack Table, charts (Recharts)
Backend API	Node.js LTS + Express + TypeScript, Zod validation, pino logging
Database	PostgreSQL + Prisma ORM, pg_trgm + full-text search
Queue / jobs	pg-boss (Postgres-based) or BullMQ + Redis; worker as separate process
Auth	Argon2/bcrypt, JWT access + rotating opaque refresh tokens, TOTP 2FA for admin
Payments	Razorpay (India), Stripe or Razorpay International (abroad), webhooks as source of truth
Storage / CDN	Cloudflare R2 + Cloudflare CDN/WAF/DNS
Messaging	Brevo/Resend (email), MSG91 (SMS, DLT), WATI/Interakt (WhatsApp)
Shipping	Shiprocket or Delhivery API (or manual tracking entry)
Hosting	Vercel (frontends), Railway/Render (API, worker, Postgres)
Monitoring	Sentry, UptimeRobot, GA4, Hotjar (optional)
Testing	Vitest/Jest, Supertest, Playwright, k6
DevOps	GitHub Actions, Docker, Turborepo + pnpm workspaces

Conventions used in the schemas

- PostgreSQL column types. Primary keys are UUID unless stated otherwise (orders also get a human order_number).
- All money is stored as INTEGER paise (never float). Use BIGINT for line and order totals.
- All timestamps are TIMESTAMPTZ in UTC; convert to IST only in the response/UI layer.
- GST rates are stored in basis points (18% = 1800). HSN code is a string.
- Soft delete with a nullable deleted_at wherever history matters. Orders store product snapshots so old invoices never change.
- Servers never trust the client for price, total, role or user id.

Things to verify before you promise them to the client

- International payments: confirm that Stripe can onboard an Indian business for international cards; otherwise use Razorpay International. This changes Phase 5.7.
- GST treatment of exports, e-invoicing and e-way bill applicability: confirm with the client's CA.
- Payment limits: UPI has per-transaction limits, so very high-value orders need cards/net banking, bank transfer or advance payment (Phase 5.8).
- SMS in India needs DLT registration and WhatsApp needs approved templates; both take days.
- Legal pages and refund policy wording must be reviewed by a qualified CA/lawyer.

Phases at a glance

No.	Phase	Goal	Tasks
0	Planning, Decisions & Foundation	a clean monorepo skeleton, frozen business decisions and every external account started, so no later phase gets blocked	9
1	Authentication & Authorization	a secure identity system for customers, staff and admins - not just login/signup	9
2	Catalog (Categories, Brands, Products, Media)	all product data that a customer browses and that the admin manages, ready for SEO	7
3	Inventory & Stock Control	stock is always correct across online and offline sales - no overselling, ever	4
4	Cart, Pricing, GST & Order Creation	a server-trusted cart and an order engine that calculates every rupee on the backend	6
5	Payments, Webhooks & Refunds	money and order state always stay consistent - Razorpay for India, international gateway for abroad	8
6	Fraud Protection & High-Value Order Controls	expensive industrial orders are verified before money or goods move	4
7	GST Invoices, Policies & Compliance	every transaction produces a compliant invoice and every legal page exists before launch	4
8	Customer Portal (Next.js Frontend)	a fast, SEO-friendly, mobile-first store that makes a Rs. 50,000 purchase feel safe	13
9	Admin Portal (Next.js)	one dashboard where the owner and staff run online and offline business together	13
10	Async Jobs & Notifications	slow work happens outside the request and survives crashes	4
11	Shipping & Order Tracking	from packed box to delivered, with the customer informed at every step	4
12	Security, Caching & Performance	make it fast and hard to abuse - this is a payments site	6
13	Testing	refactor and launch with confidence (write tests along with every phase; use this phase to fill gaps)	6
14	DevOps, Deployment & Observability	ship it like a real service with monitoring and a safe release process	5
15	Data Migration, SEO Migration & Go-Live	move from the old WordPress site to the new store with zero ranking loss and zero panic	6
16	Growth Features (After Launch)	conversion, retention and SEO features, built in the order of business impact	9
17	Senior-Level Bonus (Optional)	engineering-quality extras that make the project portfolio-grade	3

Suggested build order in one line

Decisions and accounts (0) > Auth (1) > Catalog (2) > Stock (3) > Cart/GST/Orders (4) > Payments (5) > Fraud controls (6) > Invoices and policies (7) > Customer portal (8) > Admin portal (9) > Jobs and notifications (10) > Shipping (11) > Security and performance (12) > Testing (13) > Deploy (14) > Migration and go-live (15) > Growth (16) > Bonus (17).

Phase 0: Planning, Decisions & Foundation

Goal: a clean monorepo skeleton, frozen business decisions and every external account started, so no later phase gets blocked

0.1 Monorepo & project setup

Description	Create a Git repo (main + feature branches, conventional commits). Setup a pnpm workspace + Turborepo with apps/customer (Next.js 14 App Router), apps/admin (Next.js 14), apps/api (Node.js LTS + Express + TypeScript) and packages/types, packages/utils (shared TypeScript types and helpers like paise formatting and GST math). Layer the API as routes > controllers > services > repositories, plus middlewares, utils, config, jobs. Add ESLint, Prettier, husky pre-commit hook, .env.example for every app.
API Endpoints	GET /health (api)
Done When	pnpm dev starts all three apps with one command; /health returns 200 with uptime; lint passes; a type change in packages/types is visible in all apps.

0.2 Requirements freeze & business decisions

Description	Before coding, sit with the client (Unico Traders) and get written answers to every open question. Save them as /docs/decisions.md. These answers change the schema, so do this FIRST.
Validation & Rules	Decide and record: - Price visibility: are all prices public, or are some products 'Request a Quote' only? (field products.price_mode = FIXED / QUOTE_ONLY) - Are listed prices GST-inclusive or GST-exclusive? (B2B electrical usually shows ex-GST; decide once, store both base price and gst_rate) - Minimum order quantity per product? Bulk / slab pricing needed? - International selling: yes/no? 3.5% international fee acceptable? Which countries, who handles export paperwork, customs, shipping? - Shipping: self-ship or courier partner (Shiprocket / Delhivery)? Free shipping threshold? Heavy/oversized item rules? - COD allowed? (recommended: NO for high-value items) - Manual-approval threshold (proposal says Rs. 50,000) and Request-a-Quote threshold (Rs. 2,00,000)? - EMI / pay-later needed at launch? - Return/cancellation policy for industrial goods (many are non-returnable once opened). - Who will be admin, who will be staff, how many staff accounts? - Existing data: products/customers in current WooCommerce to migrate? Current hosting/DNS provider?
Done When	/docs/decisions.md is committed and approved by the client on WhatsApp/email; every question above has an answer or an explicit 'decide later' with an owner.

0.3 ER diagram & API contract

Description	Design the ER diagram for every entity using dbdiagram.io: users, addresses, categories, brands, products, product_images, product_specs, carts, cart_items, orders, order_items, payments, refunds, stock_reservations, stock_history_log, invoices, invoice_sequences, shipments, return_requests, quotes, audit_logs, webhook_logs, otp_tokens, refresh_tokens, notification_logs, reconciliation_runs, settings. Then write the endpoint list: method, access level (public / customer / staff / admin), request and response shape. Keep both in /docs.
Validation & Rules	Conventions to follow everywhere: - Money = INTEGER paise. Line totals and order totals use BIGINT (Rs. 5,00,000 x 100 items overflows INT4). - Time = TIMESTAMPTZ in UTC; convert to IST only in the response/UI layer. - Primary keys = UUID (orders also get a human-friendly order_number like UT-26-000123). - GST rate stored as basis points (18% = 1800). HSN code is a string. - Soft delete via nullable deleted_at wherever history matters.
Done When	ER diagram + endpoint list committed in /docs and reviewed by you once after sleeping on it.

0.4 Design system & wireframes (Figma)

Description	Design before building the UI. Make a small design system (colors from the Unico logo, type scale, spacing, buttons, inputs, cards, table, badges, toasts) and low/mid-fidelity wireframes for: Home, Category listing, Product detail, Cart, Checkout, Order success, My Account, Order detail, Admin dashboard, Admin product form, Admin order detail, Admin manual-order screen. Mobile first - most Indian B2B buyers browse on phone.
UI / Screens	Set up Tailwind theme tokens + shadcn/ui from the Figma tokens so design and code stay in sync.
Done When	Wireframes for all screens listed above exist; client has approved Home + Product Detail + Checkout.

0.5 Config & environment validation

Description	Config module that reads every env variable (DB URL, JWT secrets, Razorpay keys + webhook secret, Stripe keys, MSG91, Brevo, WATI, R2 credentials, Sentry DSN, allowed origins) and validates them at startup with Zod. Missing/invalid variable = crash on boot with a clear message.
Validation & Rules	JWT secrets min 32 chars. PORT numeric. Different key sets for dev / staging / prod. Never log secret values. Never commit .env.
Done When	Removing a required env var stops the API on boot.

0.6 Validation, error handling & response format

Description	Reusable Zod validation middleware for body, query, params (collect ALL errors). Custom error classes (Validation 400, Unauthorized 401, Forbidden 403, NotFound 404, Conflict 409, TooManyRequests 429) and ONE global error handler. Wrap async handlers. One response format: success = {success, data, meta}; error = {success:false, error:{code, message, details}}.
Validation & Rules	Reject unknown fields (mass-assignment protection). Trim strings, lowercase emails. Never send stack traces in production. Unknown errors = generic 500 + logged with request-id.
Done When	Invalid input never reaches a controller; every failure has the same shape; an unexpected throw returns 500 without leaking internals.

0.7 Logging, request-id & Swagger

Description	pino structured logs with a request-id per request (accept x-request-id or generate). Log method, path, status, latency. Serve OpenAPI/Swagger at /docs and update it for every endpoint you build. Connect Sentry (free) for error tracking.
API Endpoints	GET /docs (Swagger UI)
Validation & Rules	Mask passwords, tokens, authorization header, OTPs, card/payment data, webhook payloads' sensitive fields.
Done When	One request can be traced across logs by request-id; Swagger lists every built endpoint.

0.8 Database, Prisma, migrations & seeders

Description	PostgreSQL + Prisma: connection pool, migration workflow, seed scripts (roles, one admin, demo categories, 10 demo products, states list with GST state codes, default settings). Graceful shutdown on SIGTERM.
Validation & Rules	Migrations are forward-only and committed. Seeds are idempotent (re-runnable).
Database	roles / enum: CUSTOMER, STAFF, ADMIN settings (key PK, value JSONB): store GST number, registered state code, address, low-stock default, manual-review threshold paise, quote threshold paise, international fee bps (350), invoice prefix.
Done When	Fresh clone: migrate + seed gives a working DB; Ctrl+C exits without hanging connections.

0.9 Accounts & infrastructure bootstrap (start the slow things NOW)

Description	Several things have waiting time. Start them in week 1 so they are ready when you need them: - Cloudflare account, add unicotrader.com (do NOT switch DNS yet) - Google Workspace business email (orders@unicotrader.com) - needed for SPF/DKIM/DMARC - Razorpay account + KYC (PAN, GST certificate, cancelled cheque) - 1-3 working days, test mode keys immediately - Stripe / international gateway: verify early whether Stripe supports onboarding an Indian business for international card acceptance; if not, plan Razorpay International Payments as the alternative. This can change the whole international design. - MSG91: India SMS needs DLT registration (entity + templates) - takes days, start now - WATI/Interakt + Meta Business verification for WhatsApp notifications - takes days - Brevo (email), Vercel, Railway, Cloudflare R2 bucket, Sentry, UptimeRobot, GitHub org - Delhivery/Shiprocket account if shipping integration is decided
Validation & Rules	Store credentials in a password manager; give the client ownership of gateway/DNS/Workspace accounts (you are the developer, they are the merchant).
Done When	Test-mode keys for Razorpay, MSG91 template IDs, Brevo key, R2 bucket and staging URLs exist; a checklist in /docs/infra.md shows status of each slow item.

Phase 1: Authentication & Authorization

Goal: a secure identity system for customers, staff and admins - not just login/signup

1.1 Customer register (email + phone)

Description	Customer registers with name, email, phone, password and confirm password. Hash with argon2/bcrypt, create user with role CUSTOMER, email_verified = false, phone_verified = false. Generate an email verification token and a phone OTP (SMS goes through the queue in Phase 10; until then log them in dev).
API Endpoints	POST /api/v1/auth/register Body: {full_name, email, phone, password, confirm_password, company_name?, gst_number?}
Validation & Rules	- full_name 2-100 chars. Email valid, lowercased, max 254. Phone = valid Indian 10-digit (+91) or E.164 for international. - Password 8-64 chars, upper, lower, number, special; must not contain email/phone. - Optional B2B fields: company_name, GSTIN (validate the 15-char GSTIN format; it enables ITC invoices later). - Duplicate email/phone -> 409 (and DB UNIQUE constraints for race conditions).
Database	users: id UUID PK, full_name, email UNIQUE, phone UNIQUE, password_hash NULL, role, email_verified, phone_verified, country CHAR(2) DEFAULT 'IN', company_name NULL, gstin NULL, is_active, failed_login_attempts, locked_until, last_login_at, created_at, updated_at, deleted_at.
Done When	Valid registration creates a user; every invalid case gives field-wise errors; password is never stored or logged in plain text.

1.2 OTP service (phone + email)

Description	Reusable OTP service: generate 6-digit OTP, store only the hash, expiry 5 minutes, max 5 attempts, resend cooldown 30s, max 5 OTPs/hour per phone. Used for registration, login-by-OTP, checkout verification of high-value orders and delivery OTP later.
API Endpoints	POST /api/v1/auth/otp/send Body: {phone, purpose} POST /api/v1/auth/otp/verify Body: {phone, purpose, otp}
Validation & Rules	Constant-time compare. Same response whether or not the phone exists (no enumeration). Block after max attempts. In dev/test use a fixed OTP via env flag - never in production.
Database	otp_tokens: id, user_id NULL, phone, otp_hash, purpose (REGISTER/LOGIN/CHECKOUT/DELIVERY), expires_at, attempts, used_at, created_at. Index (phone, purpose).
Done When	Correct OTP verifies once; expired/used/over-attempted OTP is rejected; rate limit works.

1.3 Login (password and OTP)

Description	Login with email + password (and optional phone OTP login). Same generic 'Invalid credentials' message for wrong user or wrong password. Lock the account for 15 minutes after 5 failed attempts. Return access token + refresh token.
API Endpoints	POST /api/v1/auth/login Body: {identifier, password}
Validation & Rules	Unverified email/phone -> 403 EMAIL_NOT_VERIFIED. Inactive/soft-deleted user cannot login. On success reset failed attempts, update last_login_at. Response time similar for existing and non-existing users.
Done When	Login works with email and phone; brute force locks the account.

1.4 Access & refresh tokens with rotation

Description	Short-lived access JWT (15 min, contains user id + role) and a long-lived opaque refresh token (7-30 days). Store only SHA-256 hash of the refresh token with device info. On refresh ROTATE: revoke old token, issue new one in the same family. If a revoked token is reused, revoke the whole family. Send refresh token in an httpOnly, Secure, SameSite cookie. Note: customer app may use NextAuth.js session on top of this; keep ONE source of truth - the API.
API Endpoints	POST /api/v1/auth/refresh
Validation & Rules	Never accept alg:none. Cookie flags httpOnly, Secure, SameSite=Lax/Strict.
Database	refresh_tokens: id, user_id FK CASCADE, token_hash CHAR(64) UNIQUE, family_id, user_agent, ip_address INET, expires_at, revoked_at, replaced_by, created_at. Indexes (user_id), (family_id).
Done When	Old refresh token stops working after one use; replay kills the whole family.

1.5 Logout, logout-all, forgot & reset password

Description	Logout revokes the current refresh token (idempotent). Logout-all revokes every token of the user. Forgot-password always returns the same response; reset-password takes token + new password + confirm, updates hash, revokes all refresh tokens, resets failed attempts.
API Endpoints	POST /api/v1/auth/logout, /logout-all POST /api/v1/auth/forgot-password {email} POST /api/v1/auth/reset-password {token, new_password, confirm_password}
Validation & Rules	Reset token expiry 30 min, single use, stored hashed. New password must differ from old.
Database	one_time_tokens: id, user_id, type (EMAIL_VERIFY/PASSWORD_RESET), token_hash UNIQUE, expires_at, used_at, created_at.
Done When	Cannot learn which emails exist; old sessions die after reset.

1.6 Email verification

Description	One-time link after registration; verifying sets email_verified = true. Resend API limited to 3/hour per email with identical response every time.
API Endpoints	POST /api/v1/auth/verify-email {token} POST /api/v1/auth/resend-verification {email}
Validation & Rules	Token expiry 24h, single use. Already verified -> 200 no-op.
Done When	Valid token verifies once; expired/used token rejected.

1.7 Profile, addresses & GST details

Description	Customer views/updates profile, manages multiple addresses (shipping and billing), saves company GSTIN for B2B invoices. Changing email requires re-verification; changing phone requires OTP.
API Endpoints	GET/PATCH /api/v1/users/me POST /api/v1/users/me/change-password GET/POST /api/v1/users/me/addresses, PATCH/DELETE /api/v1/users/me/addresses/:id
Validation & Rules	Whitelist updatable fields (never role, is_active, email_verified). Pincode 6 digits and state must be a valid Indian state (needed for GST). International addresses allow country + free-form state. Max 10 addresses.
Database	addresses: id, user_id FK, label, full_name, phone, line1, line2, city, state_code, pincode, country CHAR(2), is_default_shipping, is_default_billing, gstin NULL, created_at, deleted_at.
Done When	User cannot escalate role or edit other fields through PATCH.

1.8 Admin & staff authentication

Description	Staff/admin login is a separate flow on admin.unicotrader.com: email + password (no public registration), mandatory TOTP 2FA (Google Authenticator) for ADMIN, optional for STAFF, shorter sessions (e.g. 8h), IP/device logging. Admin creates staff accounts from the admin portal.
API Endpoints	POST /api/v1/admin/auth/login POST /api/v1/admin/auth/2fa/setup, /2fa/verify POST/PATCH /api/v1/admin/staff
Validation & Rules	Admin cookie scoped to admin domain only. Disabled staff = tokens revoked immediately. Seed the first admin from env/CLI, never from a public endpoint.
Database	users.role + admin_2fa (user_id, secret_encrypted, enabled_at, recovery_codes_hash).
Done When	Staff cannot log in on customer app with admin rights and vice versa; admin without 2FA cannot access protected routes.

1.9 RBAC & ownership middleware

Description	authenticate middleware (verify token, attach user) and authorize(...roles) middleware (CUSTOMER / STAFF / ADMIN). Permissions matrix: Staff can manage orders and stock but cannot see payments totals, reports, settings or delete products (customizable later, Phase 16 full RBAC). Ownership checks live in the service layer.
Validation & Rules	No token -> 401. Wrong role -> 403. Another user's resource -> 404 (do not reveal existence). Never trust role/user id from request body.
Done When	Customer A cannot read Customer B's order or address; staff cannot hit admin-only routes.

Phase 2: Catalog (Categories, Brands, Products, Media)

Goal: all product data that a customer browses and that the admin manages, ready for SEO

2.1 Categories & brands

Description	Admin manages a category tree (e.g. Switchgear > Industrial Switches) and brands. Slugs are unique and used in URLs. Public API returns only active categories.
API Endpoints	GET /api/v1/categories (tree) POST/PATCH/DELETE /api/v1/admin/categories GET /api/v1/brands, admin CRUD for brands
Validation & Rules	Max depth 3. Cannot delete a category that has products (409). Slug auto-generated, editable, unique. Reordering via sort_order.
Database	categories: id, parent_id NULL self FK, name, slug UNIQUE, description, image_url, sort_order, is_active, seo_title, seo_description, deleted_at. brands: id, name, slug UNIQUE, logo_url, is_active.

Done When	Category tree API returns nested JSON in one query; deleting a non-empty category is blocked.

2.2 Products (core CRUD)

Description	Admin creates/edits products: name, SKU, brand, category, short + long description, base price, MRP, GST rate, HSN code, unit, MOQ, weight/dimensions (for shipping), warranty, price_mode, status. Public APIs expose only ACTIVE products.
API Endpoints	POST/PATCH/DELETE /api/v1/admin/products GET /api/v1/admin/products/:id
Validation & Rules	- SKU unique, uppercase, 3-40 chars. - price_paise > 0 and <= MRP. gst_rate_bps from allowed set (0, 500, 1200, 1800, 2800). - HSN 4-8 digits. - price_mode FIXED or QUOTE_ONLY (QUOTE_ONLY products cannot be added to cart). - requires_manual_approval boolean for high-risk items. - Soft delete only; orders keep a snapshot so old orders never break.
Database	products: id UUID, sku UNIQUE, name, slug UNIQUE, brand_id, category_id, short_desc, description, price_paise BIGINT, mrp_paise BIGINT, gst_rate_bps INT, hsn_code, unit, min_order_qty, price_mode, requires_manual_approval, weight_grams, length_mm, width_mm, height_mm, warranty_months, status (DRAFT/ACTIVE/INACTIVE), is_featured, stock_qty INT, low_stock_threshold INT, seo_title, seo_description, created_at, updated_at, deleted_at. Indexes: (category_id, status), (brand_id), GIN on search_vector.
Done When	Product created in admin appears on the public API only when ACTIVE; deleting does not break past orders.

2.3 Product images & files (Cloudflare R2)

Description	Multiple images per product plus PDF datasheets/manuals. Use presigned upload URLs straight from admin to R2 (files never pass through the API). Generate thumbnails/WebP variants. Reorder images, mark primary.
API Endpoints	POST /api/v1/admin/uploads/presign {type, filename, contentType, size} POST/DELETE /api/v1/admin/products/:id/images, PATCH reorder
Validation & Rules	Whitelist types (jpg, png, webp, pdf). Max 5 MB image / 10 MB PDF. Random object keys (never user filenames). Validate dimensions >= 1000x1000 as warning. Serve via media.unicotrader.com through Cloudflare.
Database	product_images: id, product_id, url, alt_text, sort_order, is_primary. product_files: id, product_id, title, url, type (DATASHEET/MANUAL/CERTIFICATE).
Done When	Admin uploads 5 images, reorders them and the public page shows the new order; oversized/invalid files are rejected.

2.4 Specs, variants & bulk price slabs

Description	Technical specs as key-value rows (voltage, current rating, phase, IP rating...) shown in a table on the product page. Decide variants: simple approach = each variant is its own product linked by a group id (recommended for MVP). Optional quantity slabs (buy 10+ get lower unit price) for B2B.
Validation & Rules	Spec keys come from a managed list per category so filters stay consistent. Slabs must not overlap; price decreases as quantity increases.
Database	product_specs: id, product_id, key, value, sort_order. product_groups: id, name (variant family). price_slabs: id, product_id, min_qty, price_paise.
Done When	A product page can show a specs table; slab price is picked correctly by quantity on the server.

2.5 Public listing, search & filters

Description	Public product listing with category, brand, price range, in-stock filter, spec filters, sorting, pagination. Search by name/SKU/brand with PostgreSQL full-text search (tsvector + GIN) and pg_trgm for typo tolerance.
API Endpoints	GET /api/v1/products?category=&brand=&minPrice=&maxPrice=&inStock=&q=&sort=&page=&limit= GET /api/v1/products/:slug GET /api/v1/search/suggest?q=
Validation & Rules	limit 1-50 (default 12). sort from a whitelist (price_asc, price_desc, newest, popular). q min 2 chars. Never return draft/inactive products or cost fields. Total count in meta.
Done When	Searching a misspelled product name still finds it; listing responds in under 200 ms locally with 1,000 products.

2.6 Bulk import / export (CSV, Excel)

Description	Admin uploads an Excel/CSV of products (the client's existing data goes here) with a dry-run preview showing row-wise errors, then commits. Export all products to Excel. Image URLs/zips handled in a second pass.
API Endpoints	POST /api/v1/admin/products/import?dryRun=true|false GET /api/v1/admin/products/export
Validation & Rules	Import runs in a background job for big files. Upsert by SKU. Report: created / updated / failed rows with reasons, downloadable. Never partially corrupt - validate all rows first.
Done When	A 500-row file imports in one go; bad rows are reported, good rows are not blocked (or all-or-nothing mode chosen).

2.7 SEO data & structured data

Description	Each product/category has SEO title, description, canonical URL, Open Graph image. Generate sitemap.xml and robots.txt. Add JSON-LD Product schema (name, image, sku, brand, offers with price, availability) - required for Google Shopping / Merchant Center.
Validation & Rules	Clean URLs: /products/[slug], /category/[slug]. Pagination uses rel canonical. Out-of-stock products stay indexed with correct availability.
Done When	Google Rich Results Test passes for a product page; sitemap lists all active products.

Phase 3: Inventory & Stock Control

Goal: stock is always correct across online and offline sales - no overselling, ever

3.1 Stock model & history log

Description	products.stock_qty is the live number. Every change (online sale, offline sale, manual adjustment, restock, return, cancellation) must go through ONE stock service function that updates the number AND writes a stock_history_log row in the same transaction. Nobody updates stock_qty directly.
Validation & Rules	Never allow stock_qty < 0 (DB CHECK + service check). reason enum: ONLINE_SALE, OFFLINE_SALE, MANUAL_ADJUST, RESTOCK, RETURN, ORDER_CANCEL, RESERVATION_RELEASE. Record admin_id and reference id.
Database	stock_history_log: id BIGSERIAL, product_id, change_qty INT, qty_after INT, reason, reference_type, reference_id, admin_id NULL, note, created_at. Index (product_id, created_at DESC).
Done When	Sum of history rows equals current stock for any product (write a check script).

3.2 Stock reservation at checkout

Description	When the customer starts payment, reserve stock for 15 minutes so two buyers cannot pay for the last unit. Atomic conditional update: reserve only where available_qty >= requested. available = stock_qty - active reservations.
Validation & Rules	Lock rows in sorted product-id order to avoid deadlocks. All-or-nothing across the cart. Expiry job releases reservations; lazy expiry check also inside checkout so correctness never depends only on cron. On payment confirmed convert reservation to permanent deduction.
Database	stock_reservations: id, order_id, product_id, qty, status (ACTIVE/CONSUMED/RELEASED/EXPIRED), expires_at, created_at. Index (status, expires_at).
Done When	50 parallel checkouts for the last unit -> exactly 1 success; no reservation stays ACTIVE after expiry even with the cron stopped.

3.3 Low-stock alerts & out-of-stock behaviour

Description	When stock drops to or below low_stock_threshold, raise an alert (admin dashboard widget + email/WhatsApp to admin, once per crossing). Out-of-stock products automatically show 'Out of Stock' with an optional 'Notify me when available' form.
API Endpoints	GET /api/v1/admin/inventory/low-stock POST /api/v1/products/:id/notify-me
Validation & Rules	Do not spam: one alert per threshold crossing, reset when restocked above threshold.
Done When	Selling the unit that hits threshold triggers exactly one alert.

3.4 Manual stock adjustment & stock screens API

Description	Admin adds stock (new purchase), removes stock (damage/loss) or sets an absolute count after a physical stock-take. Every adjustment needs a reason and note.
API Endpoints	POST /api/v1/admin/inventory/adjust {productId, delta|setTo, reason, note} GET /api/v1/admin/inventory/history?productId=&from=&to=
Validation & Rules	Staff can adjust only if permission granted. Adjustments over a limit are flagged in audit log.
Done When	History shows who changed what, when and why.

Phase 4: Cart, Pricing, GST & Order Creation

Goal: a server-trusted cart and an order engine that calculates every rupee on the backend

4.1 Cart API

Description	Logged-in customers have a server-side cart; guests use a local cart that merges into the server cart on login. Add, update quantity, remove, clear. Cart always returns freshly priced lines.
API Endpoints	GET /api/v1/cart POST /api/v1/cart/items {productId, qty} PATCH/DELETE /api/v1/cart/items/:id
Validation & Rules	Reject QUOTE_ONLY products. qty >= min_order_qty and <= available stock and <= 999. Price shown is always recomputed from DB (never trust client). Show stock warnings, do not silently drop lines.
Database	carts: id, user_id UNIQUE, updated_at. cart_items: id, cart_id, product_id, qty, UNIQUE(cart_id, product_id).
Done When	Changing a product price in admin is reflected in every cart on next fetch.

4.2 Pricing & GST engine

Description	One pure, well-tested function: calculatePricing(lines, shippingAddress, customerType) returning per-line taxable value, CGST/SGST or IGST, shipping, international fee, rounding and grand total - all in paise. Seller state = Gujarat: if shipping state = Gujarat charge CGST + SGST (half each), otherwise IGST. Export orders: GST treatment (zero-rated with LUT or IGST paid) must be confirmed with the client's CA.
Validation & Rules	- Round per line, sum lines (be consistent with invoice). - Show 'Inclusive/Exclusive of GST' exactly as decided in 0.2. - International fee = 3.5% (350 bps) of order value, shown as a separate line at checkout and stored on the order. - Shipping from weight/zone table or courier rate API. - Never compute totals in the frontend except for display preview.
Done When	Table-driven unit tests with 20+ cases (intra-state, inter-state, international, slab pricing, rounding) all pass.

4.3 Order schema & state machine

Description	Define the order lifecycle in one central function that rejects invalid transitions. Statuses: PENDING_PAYMENT -> PENDING_REVIEW (high-value) -> CONFIRMED -> PROCESSING -> PACKED -> SHIPPED -> OUT_FOR_DELIVERY -> DELIVERED; side exits CANCELLED, EXPIRED; payment_status tracked separately: UNPAID, PAID, PARTIALLY_REFUNDED, REFUNDED.
Validation & Rules	Invalid transition -> 409. Every transition writes an audit log + order_status_history row. Order stores snapshots of product name, SKU, HSN, GST rate and unit price at time of purchase (so later edits never change old invoices).
Database	orders: id UUID, order_number UNIQUE, user_id NULL (walk-in offline orders may have no account), order_source (ONLINE/OFFLINE), status, payment_status, currency CHAR(3), subtotal_paise, gst_paise BIGINT, shipping_paise, intl_fee_paise, discount_paise, total_paise BIGINT, is_international, shipping_address JSONB, billing_address JSONB, customer_gstin, idempotency_key, UNIQUE(user_id, idempotency_key), created_by_admin_id NULL, placed_at, expires_at, notes_internal, created_at, updated_at. order_items: id, order_id, product_id, sku, name, hsn_code, qty, unit_price_paise, gst_rate_bps, gst_amount_paise, line_total_paise. order_status_history: id, order_id, from_status, to_status, actor_id, note, created_at.
Done When	Transition function has tests for every allowed and forbidden move.

4.4 Checkout API (create order + reserve stock)

Description	Checkout validates cart, address and country, runs pricing, applies high-value rules (Phase 6), reserves stock and creates the order in ONE transaction. Returns order id + amounts. Same Idempotency-Key returns the same order.
API Endpoints	POST /api/v1/checkout Header: Idempotency-Key Body: {shippingAddressId, billingAddressId?, gstin?, notes?}
Validation & Rules	Cart must not be empty. Email AND phone verified. If any line is unavailable -> 409 with the list of lines, order not created. Total from server only. Two parallel requests with the same key create ONE order.
Done When	Parallel duplicate checkout calls produce one order and one reservation set.

4.5 Order expiry & cancellation by customer

Description	Unpaid orders expire after the reservation time (15-30 min): status EXPIRED, stock released. Customers can cancel PENDING_PAYMENT orders themselves; after payment, cancellation is a request handled via Phase 7/8 return flow depending on status (not allowed after SHIPPED).
API Endpoints	POST /api/v1/orders/:id/cancel GET /api/v1/orders, GET /api/v1/orders/:id
Validation & Rules	Customer sees only own orders (404 otherwise). Release must be idempotent and transactional. Never release stock of a PAID order without a cancellation record.
Done When	Another user's order id returns 404; expired order frees stock with cron stopped (lazy check).

4.6 Request-a-Quote flow

Description	For QUOTE_ONLY products and for carts above the quote threshold, the customer submits a quote request instead of paying. Admin reviews, sets final prices/shipping/validity, sends the quote; customer accepts and pays through a payment link tied to a normal order.
API Endpoints	POST /api/v1/quotes {items[], message, company, gstin} GET /api/v1/quotes/:id POST /api/v1/quotes/:id/accept Admin: GET/PATCH /api/v1/admin/quotes, POST /api/v1/admin/quotes/:id/send
Validation & Rules	Quote has validity (e.g. 7 days) and locked prices. Accepting converts to an order with those prices; expired quotes cannot be accepted.
Database	quotes: id, quote_number, user_id, status (REQUESTED/QUOTED/ACCEPTED/EXPIRED/REJECTED), valid_until, total_paise, admin_notes, created_at. quote_items: id, quote_id, product_id, qty, quoted_unit_price_paise.
Done When	Request -> admin quote -> customer accepts -> pays works end to end.

Phase 5: Payments, Webhooks & Refunds

Goal: money and order state always stay consistent - Razorpay for India, international gateway for abroad

5.1 Create Razorpay order

Description	For an own PENDING_PAYMENT, non-expired order create a Razorpay order for the total in paise and return order id + public key. One active payment per order; calling again returns the same Razorpay order. Use Razorpay hosted/standard Checkout so card data never touches your server.
API Endpoints	POST /api/v1/orders/:id/payments
Validation & Rules	Amount always from DB. Booking-style idempotency. If order total exceeds the method limit (UPI ~Rs. 1,00,000), show only suitable methods (net banking, cards, bank transfer) or push the customer to a bank-transfer/RTGS flow (see 5.8).
Database	payments: id, order_id, gateway (RAZORPAY/STRIPE/CASH/UPI_MANUAL/CARD_MANUAL/BANK_TRANSFER), provider_order_id UNIQUE, provider_payment_id UNIQUE NULL, amount_paise BIGINT, currency, status (CREATED/AUTHORIZED/CAPTURED/FAILED/REFUNDED/PARTIALLY_REFUNDED), method, offline_reference (UTR / card txn id), failure_reason, recorded_by_admin_id NULL, created_at, updated_at. payment_events: id, payment_id NULL, provider_event_id UNIQUE, event_type, payload JSONB, signature_valid, processed_at, created_at.
Done When	Order created once per order; amount equals order total.

5.2 Client-side verification

Description	After checkout the client sends razorpay_order_id, payment_id and signature. Verify the HMAC on the server for instant UX feedback. The webhook stays the source of truth for confirming the order.
API Endpoints	POST /api/v1/payments/verify {razorpay_order_id, razorpay_payment_id, razorpay_signature}
Validation & Rules	Constant-time comparison. Signature mismatch -> 400. Order must belong to the logged-in user.
Done When	Tampered signature rejected.

5.3 Razorpay webhook handler

Description	Raw-body endpoint verifying X-Razorpay-Signature. Save every event in payment_events (unique event id = dedupe). On payment.captured, in one transaction: payment = CAPTURED, order.payment_status = PAID, order.status = CONFIRMED (or PENDING_REVIEW for high value), reservation consumed, stock deducted, invoice job queued. Respond 200 fast; slow work goes to the queue.
API Endpoints	POST /api/v1/webhooks/razorpay
Validation & Rules	Invalid signature -> 400 + log. Duplicate event -> 200, do nothing. Unknown order -> log + 200. Out-of-order events must never move a payment backwards (FAILED after CAPTURED). Webhook IP/secret hygiene.
Done When	Replaying the same webhook 3 times confirms the order once.

5.4 Failure scenarios & late payments

Description	Implement and test each scenario: (a) money debited but status pending - keep order PENDING_PAYMENT, reconcile; (b) double click/duplicate payment - idempotency + unique payment_id; (c) user closed the tab mid-payment - webhook still confirms; (d) webhook missed - polling job; (e) gateway timeout - reuse same Razorpay order on retry; (f) payment captured after order expired or stock gone - auto refund + admin alert; (g) payment.failed - release or allow retry while reservation active.
Validation & Rules	Never confirm an order whose stock reservation no longer belongs to it. Late capture = refund, not oversell.
Done When	Every scenario above has an automated test and ends with consistent order + stock + money.

5.5 Reconciliation jobs

Description	Job A (every 15-30 min): fetch recent Razorpay payments and fix any PENDING order whose payment is captured. Job B (daily): compare Razorpay settlement/payment report with DB totals; mismatches go into reconciliation_runs and alert the admin.
Validation & Rules	Jobs are idempotent and safe to run concurrently (unique job ids / advisory lock).
Database	reconciliation_runs: id, run_date, gateway, total_checked, mismatches INT, details JSONB, created_at.
Done When	Simulate a missed webhook: the 15-minute job confirms the order automatically.

5.6 Refunds & cancellation policy

Description	Admin initiates full or partial refund (item-level) from the order screen; system calls Razorpay refund API and updates status via refund webhooks. Status: INITIATED -> PROCESSED / FAILED. Refund policy table comes from 0.2 (e.g. before dispatch = 100%, after dispatch = return flow, opened electrical goods = restocking fee).
API Endpoints	POST /api/v1/admin/orders/:id/refund {amountPaise, itemIds?, reason} GET /api/v1/orders/:id/refunds
Validation & Rules	Refund <= captured - already refunded. One refund request id per action (idempotent). Order payment_status becomes PARTIALLY_REFUNDED / REFUNDED. Stock returns only if admin marks items as restockable.

Database	refunds: id, payment_id, order_id, provider_refund_id UNIQUE NULL, amount_paise, status, reason, initiated_by, created_at, updated_at.
Done When	Retried refund click never refunds twice; amounts match policy exactly.

5.7 International payments & 3.5% fee

Description	Customer with non-India shipping/billing country is routed to the international gateway (Stripe if onboarding is possible, else Razorpay International). Add a transparent 3.5% international processing fee line at checkout, stored in orders.intl_fee_paise. Handle currency: show INR price plus an indicative foreign amount, but charge exactly what the gateway confirms.
API Endpoints	POST /api/v1/orders/:id/payments/international POST /api/v1/webhooks/stripe
Validation & Rules	Separate webhook secret and signature verification. Same idempotency, event dedupe and state rules as Razorpay. Document export/GST/shipping obligations from the CA before enabling.
Done When	Test-mode international payment confirms an order through its webhook; fee line shows on checkout and invoice.

5.8 High-value payment fallback (bank transfer / advance)

Description	For orders above method limits or the client's comfort level: allow 50% advance online + balance before dispatch, or bank transfer (NEFT/RTGS) with UTR submission verified by admin (or Razorpay virtual accounts for automatic matching). Admin marks the payment received.
API Endpoints	POST /api/v1/orders/:id/payments/bank-transfer {utr, amount, proofUrl} Admin: POST /api/v1/admin/payments/:id/mark-received
Validation & Rules	Order stays PENDING_PAYMENT until admin confirms. Duplicate UTR blocked (UNIQUE offline_reference).
Done When	A Rs. 3,00,000 order can be completed through bank transfer with a full audit trail.

Phase 6: Fraud Protection & High-Value Order Controls

Goal: expensive industrial orders are verified before money or goods move

6.1 Risk rules & manual review queue

Description	Orders above the manual-approval threshold (default Rs. 50,000) or containing requires_manual_approval products land in PENDING_REVIEW after payment. Admin calls the customer, verifies, then Approves (-> CONFIRMED) or Rejects (-> refund). Risk flags add reasons: new account + high value, shipping state differs from billing, repeated failures, mismatched phone/email domains, many orders in short time.
API Endpoints	GET /api/v1/admin/orders/review-queue POST /api/v1/admin/orders/:id/approve, /reject
Validation & Rules	Reject requires a reason and auto-starts refund. Review decisions are audit logged with actor + IP. SLA timer shows how long an order has waited.
Database	order_risk_flags: id, order_id, code, score, details JSONB, created_at.
Done When	A Rs. 75,000 order waits in the queue; approve moves it forward, reject refunds it.

6.2 Velocity checks & OTP at checkout

Description	Mandatory phone OTP before placing an order above a limit. 3+ failed payments in 10 minutes -> temporary block + admin alert. Rate limit order creation per user and per IP.
Validation & Rules	Blocks are time-bound and visible to admin. Do not leak the reason in the public error message.
Done When	Scripted repeated failures trigger the block; normal users are unaffected.

6.3 Delivery OTP & proof of delivery

Description	High-value shipments need proof: delivery OTP sent to the customer's phone and entered by the courier/staff, or signature + photo uploaded by the courier partner/admin. Stored as dispute evidence.
API Endpoints	POST /api/v1/admin/shipments/:id/delivery-proof {otp?|photoUrl, signatureUrl?}
Database	shipment_proofs: id, shipment_id, type (OTP/PHOTO/SIGNATURE), url_or_value_hash, captured_at, captured_by.
Done When	Order cannot be set DELIVERED for high-value items without proof.

6.4 Chargeback / dispute evidence pack

Description	One admin action collects the evidence for a dispute: order, invoice, payment events, delivery proof, communication logs, OTP confirmation logs - exported as a ZIP/PDF ready to upload to Razorpay.
API Endpoints	GET /api/v1/admin/orders/:id/evidence-pack
Done When	Pack generated for a sample order contains all items above.

Phase 7: GST Invoices, Policies & Compliance

Goal: every transaction produces a compliant invoice and every legal page exists before launch

7.1 GST invoice generation (PDF)

Description	Generate a GST tax invoice PDF for every paid online order and every offline order: seller details + GSTIN, buyer details (+GSTIN if given), invoice number and date, place of supply, line items with HSN, taxable value, CGST/SGST/IGST per line, shipping, grand total in numbers and words, terms, QR. Store the PDF in R2 and link it to the order.
API Endpoints	GET /api/v1/orders/:id/invoice (customer, own) GET /api/v1/admin/orders/:id/invoice POST /api/v1/admin/orders/:id/invoice/regenerate (credit-note rules apply)
Validation & Rules	Invoice numbers are sequential per financial year with no gaps and max 16 characters (GST rule), e.g. UT/26-27/000123. Allocate numbers inside a transaction using a locked sequence row. Invoices are immutable once issued - corrections are credit/debit notes. Generate in a background job so webhooks stay fast. Confirm e-invoicing/e-way bill applicability with the client's CA.
Database	invoices: id, order_id UNIQUE, invoice_number UNIQUE, financial_year, issued_at, pdf_url, taxable_paise, cgst_paise, sgst_paise, igst_paise, total_paise, buyer_gstin. invoice_sequences: financial_year PK, last_number.
Done When	Two orders paid at the same time get consecutive unique numbers; PDF totals match order totals to the paisa.

7.2 Credit notes for refunds & returns

Description	When a refund relates to taxable goods, create a credit note referencing the original invoice (own sequence, e.g. CN/26-27/000001) with tax breakup.
Database	credit_notes: id, invoice_id, refund_id, number UNIQUE, amount_paise, tax_breakup JSONB, pdf_url, created_at.
Done When	Partial refund produces a correct credit note PDF.

7.3 Policy & legal pages + grievance officer

Description	Pages: Privacy Policy, Terms & Conditions, Return/Refund/Cancellation Policy, Shipping Policy, Warranty Policy, Contact + Grievance Officer (name, email, phone, response time as per Consumer Protection (E-Commerce) Rules 2020). Show GSTIN, registered address and 'Est. 1965' in footer. Content is editable in admin (Phase 9 CMS-lite) and versioned.
Validation & Rules	Draft with the client; final wording must be reviewed by a qualified CA/lawyer. DPDP Act 2023: collect consent for data use, provide data deletion/export request path.
Done When	All pages live and linked from footer and checkout (T&C checkbox required before payment).

7.4 Consent, cookies & data rights

Description	Cookie/consent banner (analytics, marketing pixels load only after consent), checkbox consent on registration/checkout, 'request my data' and 'delete my account' flows (soft delete + anonymise where invoices must be retained by law).
API Endpoints	POST /api/v1/users/me/data-request {type} DELETE /api/v1/users/me
Validation & Rules	Invoices and order records are retained as legally required; personal fields in non-legal tables are anonymised.
Done When	Deleting an account anonymises profile but keeps invoice data intact.

Phase 8: Customer Portal (Next.js Frontend)

Goal: a fast, SEO-friendly, mobile-first store that makes a Rs. 50,000 purchase feel safe

8.1 App shell, layout & design system

Description	Next.js 14 App Router with Tailwind + shadcn/ui. Header (logo, search, category menu, cart, account, WhatsApp), footer (policies, GSTIN, address, trust badges), responsive navigation, loading skeletons, error and 404 pages, toast system, API client with token refresh handling, Zustand store for cart state.
UI / Screens	Components: Button, Input, Select, Modal, Drawer (mobile menu/cart), Badge, Breadcrumb, Pagination, Price, StockBadge, Skeleton, EmptyState.
Validation & Rules	Server Components for SEO pages, Client Components only where interactive. Never expose secret keys; only NEXT_PUBLIC_ values. Images via next/image with R2/Cloudflare domain.
Done When	Lighthouse mobile >= 85 on an empty shell; keyboard navigation works for header and menus.

8.2 Homepage

Description	Hero banner, category tiles, featured products, brand strip, 'Est. 1965' trust section, why-buy-from-us (genuine products, GST invoice, warranty, support), testimonials placeholder, WhatsApp CTA, newsletter/enquiry capture.
UI / Screens	Content blocks fed by admin-managed banners/featured flags (hard-code first, CMS in Phase 16).
Done When	Home LCP < 2.5 s on mobile 4G test; all sections responsive.

8.3 Category & product listing pages

Description	SSR listing with filters (brand, price, availability, specs), sort, pagination (or infinite scroll with URL state), product cards (image, name, price, GST note, stock status, add to cart / request quote), breadcrumbs, empty state.
UI / Screens	Filters stored in the URL so pages are shareable and crawlable. Mobile: filters in a bottom sheet.
Done When	Applying filters updates the URL and results without full reload; back button works.

8.4 Product detail page

Description	Image gallery with zoom, price block (MRP, discount, GST info), stock status, quantity selector respecting MOQ and slabs, add to cart / buy now / request quote, specs table, description, downloads (datasheet), warranty, delivery estimate by pincode, related products, JSON-LD schema, share/WhatsApp enquiry button.
UI / Screens	Sticky mobile CTA bar (price + Add to cart).
Done When	Out-of-stock and QUOTE_ONLY products show the correct CTA; datasheet downloads work.

8.5 Search UX

Description	Header search with debounced suggestions (products, categories, brands), search results page, recent searches, 'no results' help with WhatsApp contact.
Validation & Rules	Debounce 300 ms, cancel stale requests, escape output.
Done When	Typing a partial or misspelled name shows the right product in suggestions.

8.6 Auth screens

Description	Register (email + phone + optional company/GSTIN), OTP verify, login, forgot/reset password, email verification landing page. Inline validation mirroring server rules, show password toggle, clear error messages, redirect back to the page the user came from.
Validation & Rules	Do not keep tokens in localStorage; rely on httpOnly cookies. Disable submit while loading.
Done When	A new user can register, verify phone OTP and land back on the product they were viewing.

8.7 Cart page & mini-cart

Description	Line items with image, price, quantity stepper, remove, per-line stock warnings, GST + shipping estimate, coupon box (Phase 16), 'Proceed to checkout', guest-to-login merge. Mini cart drawer from header.
Done When	Quantity changes update totals from the server response and handle stock errors gracefully.

8.8 Checkout flow

Description	Steps: address (select/add, GSTIN for business), shipping method, review (items, GST breakup, shipping, international fee line if applicable), consent checkbox, pay. Opens Razorpay Checkout; on success shows a 'verifying payment' screen that polls order status until the webhook confirms. Handles failure/retry with the SAME order.
UI / Screens	States: PAYMENT_PENDING, PAYMENT_FAILED (retry), PENDING_REVIEW (explain verification call), CONFIRMED.
Validation & Rules	Disable pay button after click. Never mark success from the client alone - wait for server status. Preserve cart if payment fails. International customers see the 3.5% fee before paying.
Done When	Full test-mode payment works; closing the tab mid-payment and returning shows the correct final status.

8.9 Order success, order history & detail

Description	Confirmation page with order number and next steps. My Orders list with status badges and filters, order detail with timeline (placed, paid, packed, shipped, delivered), items, addresses, payment info, invoice download, tracking link, cancel/return request buttons only when allowed.
Done When	Customer can find any order, download its invoice and see the live status timeline.

8.10 Account area & returns

Description	Profile, addresses, change password, saved GSTIN, notification preferences, quote requests list, return/cancellation request form with reason and photo upload, request status tracking.
Done When	Customer can raise a return request and see admin's response.

8.11 Static pages, trust & contact

Description	About (history since 1965), Contact (map, address, phone, WhatsApp, enquiry form with CAPTCHA), policy pages from Phase 7, FAQ, trust badges (Secured by Razorpay, GST invoice, genuine products).
Done When	All footer links work; enquiry form emails the team.

8.12 SEO, analytics & performance

Description	Metadata API per page, canonical URLs, sitemap + robots, JSON-LD, GA4 + Meta Pixel with consent, Search Console, image optimisation, font optimisation, route-level code splitting, caching headers via Cloudflare.
Done When	Lighthouse mobile >= 80 on Home, Listing, Product; Core Web Vitals pass in field test after launch.

8.13 Accessibility & mobile polish

Description	Test on real Android + iPhone. Tap targets >= 44px, readable contrast, form labels, focus states, no horizontal scroll, working back button in checkout, offline/slow-network messages.
Done When	Checklist signed off on 2 real devices.

Phase 9: Admin Portal (Next.js)

Goal: one dashboard where the owner and staff run online and offline business together

9.1 Admin app shell & access control

Description	Separate Next.js app on admin.unicotrader.com: login with 2FA, sidebar, top bar with user menu, role-based menu (hide what the role cannot use), breadcrumbs, global search (orders, customers, products), reusable DataTable (server-side sort/filter/pagination), forms, confirm dialogs, toasts.
Validation & Rules	Route guards on both client and API. Auto-logout on inactivity. noindex + robots disallow. Optional IP allowlist via Cloudflare Access.
Done When	Staff user sees only permitted menu items and gets 403 on forbidden URLs.

9.2 Dashboard

Description	Widgets: today's revenue (online + offline), this month with online vs offline graph, orders by status counts, failed payments, orders awaiting review, low-stock list, recent 10 orders, top-selling products this month, pending quotes.
API Endpoints	GET /api/v1/admin/dashboard/summary?from=&to=
Validation & Rules	Aggregate in SQL (not in JS). Cache for 30-60 s. Date range uses IST day boundaries.
Done When	Dashboard numbers match a manual SQL total for the same day.

9.3 Product management screens

Description	Product list (search, filters, status toggles, stock column), create/edit form with tabs (General, Pricing & GST, Inventory, Images, Specs, SEO), image uploader with reorder, duplicate product, bulk actions (activate, deactivate, change category), CSV import/export UI from Phase 2.6.
Done When	Admin can add a complete product with 5 images and it appears on the website.

9.4 Category, brand & content screens

Description	Category tree editor with drag-drop ordering, brand manager, banner/featured-section manager for the homepage, policy pages editor (versioned).
Done When	Changing a banner in admin updates the homepage after cache refresh.

9.5 Inventory screens

Description	Live stock table with filters (low, out of stock), inline adjust with reason, stock history viewer per product, stock-take mode (enter counted quantity and see differences before committing).
Done When	Every adjustment appears in history with admin name and reason.

9.6 Online order management

Description	Order list with filters (status, payment status, source, date, city), order detail with items, addresses, payment trail, status timeline, internal notes, actions (confirm, pack, ship with courier + tracking number, mark out-for-delivery, mark delivered with proof, cancel, refund), print packing slip and invoice.
Validation & Rules	Only valid next-status buttons are shown (driven by the same state machine). Every action is audit-logged.
Done When	An order can be moved from CONFIRMED to DELIVERED using only the UI, and the customer sees each step.

9.7 Manual (offline) order entry

Description	Counter/phone sale screen: pick an existing customer or create a quick customer (name + phone, GSTIN optional) or walk-in; add products with quantity (barcode/SKU search), see live GST total, choose payment mode (Cash, UPI, Card, Bank transfer) and reference (UTR/txn id), mark paid, create the order with order_source = OFFLINE. Stock is deducted immediately, invoice generated, print/share via WhatsApp. Support 'unpaid / pay later' credit orders if the client needs them.
API Endpoints	POST /api/v1/admin/orders/manual {customer|walkIn, items[], payments[], shippingAddress?, notes}
Validation & Rules	Same pricing engine as online. Admin may apply a manual discount within a permission limit (logged). Cannot sell more than stock unless admin has override permission. Duplicate-submit protection with idempotency key.
Done When	A counter sale takes under 60 seconds, reduces stock once and appears in the combined revenue report as OFFLINE.

9.8 Payments & refunds screens

Description	Payment list (online + offline), filters by gateway/status, failed and pending payments, reconcile button, refund dialog (full/partial with reason), mark bank transfer received, link to Razorpay dashboard.
Done When	Admin can refund an order partially and see status move to PROCESSED via webhook.

9.9 Review queue, returns & quotes

Description	Screens for: high-value review queue (approve/reject with call notes), return/cancellation requests (approve, schedule pickup, inspect, refund), quote requests (price items, send quote, track acceptance).
Done When	Each queue shows age, assignee and one-click actions.

9.10 Customers

Description	Customer list and detail: contact, company/GSTIN, addresses, total orders and spend, order history, internal tags (VIP, bulk buyer) and notes, block/unblock, resend verification, create order on behalf.
Done When	Searching a phone number finds both online and offline-created customers.

9.11 Reports & exports

Description	Daily/weekly/monthly revenue (online + offline), online vs offline breakdown, product-wise and category-wise sales, customer-wise purchases, GST report (taxable value, CGST/SGST/IGST by rate, B2B vs B2C) for the CA, stock valuation, payment mode report. Every report exports to Excel/CSV.
API Endpoints	GET /api/v1/admin/reports/:type?from=&to=&format=json|xlsx
Validation & Rules	Large exports run as a background job and arrive as a download link. Reports use IST month boundaries and exclude cancelled/expired orders unless stated.
Done When	GST report totals reconcile with invoice totals for the month.

9.12 Staff, roles & audit log viewer

Description	Create/disable staff, assign role (Admin/Staff), customizable permissions (orders, stock, payments view, reports, settings). Audit log viewer with filters (who, what, entity, date) and CSV export.
Database	audit_logs: id BIGSERIAL, actor_id, actor_role, action, entity_type, entity_id, old_value JSONB, new_value JSONB, ip_address INET, user_agent, created_at (append-only).
Done When	Every admin write action appears in the audit log.

9.13 Settings

Description	Business profile (name, GSTIN, address, logo), tax defaults, invoice prefix and next number, shipping rules/zones, international fee %, manual-review and quote thresholds, notification toggles/templates, payment settings (read-only keys status), maintenance mode.
Validation & Rules	Only ADMIN. Changes are audit logged and take effect without redeploy.
Done When	Changing the review threshold changes behaviour of the next order.

Phase 10: Async Jobs & Notifications

Goal: slow work happens outside the request and survives crashes

10.1 Queue & worker

Description	Run a separate worker process. Choose pg-boss (uses your existing PostgreSQL, no extra service) for the MVP, or BullMQ + Redis (Upstash) if you want its dashboard. Queues: emails, sms, whatsapp, invoices, maintenance, reports, imports. Retry 5 times with exponential backoff then dead-letter. Admin-only queue dashboard or status page.
Validation & Rules	Jobs idempotent (safe to run twice). Pass ids, not whole objects. Scheduled jobs must not run twice with multiple workers (unique job ids / singleton keys).
Done When	Killing the worker mid-job does not lose the job.

10.2 Email notifications

Description	Brevo/Resend transactional emails with templates: verify email, reset password, order placed, payment confirmed, order under review, shipped, out for delivery, delivered, cancelled, refund initiated/processed, quote sent, low-stock alert (admin), daily summary (admin). Include invoice PDF attachment on confirmation.
Validation & Rules	Never block an API response on sending. SPF/DKIM/DMARC configured for the domain.
Database	notification_logs: id, user_id NULL, type, channel (EMAIL/SMS/WHATSAPP), recipient, status (QUEUED/SENT/FAILED), attempts, error, provider_message_id, created_at, sent_at.
Done When	Confirmation email with invoice arrives after payment while the API stays fast.

10.3 SMS (MSG91) & WhatsApp

Description	OTP and order-status SMS using DLT-approved templates. WhatsApp order updates via WATI/Interakt with approved templates (order confirmed, shipped with tracking link, delivered). Respect opt-in and quiet hours for non-transactional messages.
Validation & Rules	Template variables validated before sending. Fall back to SMS/email if WhatsApp fails. Cost guard: daily max per user.
Done When	Order shipped triggers a WhatsApp message with a working tracking link.

10.4 Scheduled jobs

Description	Expire unpaid orders and reservations (every minute), payment reconciliation (15-30 min and daily), nightly cleanup of expired OTPs/tokens, abandoned-cart reminders (Phase 16), daily admin summary email, weekly database backup verification, stock-low digest.
Done When	All jobs visible in the dashboard with last-run time and result.

Phase 11: Shipping & Order Tracking

Goal: from packed box to delivered, with the customer informed at every step

11.1 Shipping rules & rate calculation

Description	Start simple: shipping zones by state/pincode with weight-based slabs and free-shipping threshold, plus 'pickup from store (Surat)'. Optionally call a courier rate API for live rates. Heavy/fragile items get special handling flags. Pincode serviceability check on the product page.
API Endpoints	GET /api/v1/shipping/estimate?pincode=&items=
Validation & Rules	Freight is calculated on the server and stored on the order. International shipping decided in 0.2 (quote-based freight is acceptable for MVP).
Database	shipping_zones, shipping_rates (zone_id, min_weight_g, max_weight_g, price_paise), shipments (id, order_id, courier, awb_number, tracking_url, status, shipped_at, delivered_at, weight_grams, charges_paise).
Done When	Estimate for a Gujarat pincode and a Delhi pincode returns different correct charges.

11.2 Courier integration (Shiprocket / Delhivery)

Description	Admin clicks 'Create shipment': API books the shipment, stores AWB + tracking URL, downloads label. Webhooks/polling update shipment status into order timeline. If the client self-ships, admin enters courier name + tracking number manually (always supported).
API Endpoints	POST /api/v1/admin/orders/:id/shipments POST /api/v1/webhooks/courier
Validation & Rules	Verify courier webhook secrets. Map courier statuses to your states in one table. Failed delivery / RTO creates an admin task.

Done When	Shipment status changes show on the customer order page automatically.

11.3 Tracking page & notifications

Description	Public/customer tracking view by order number + phone/OTP, timeline with courier checkpoints, SMS/WhatsApp/email on each major step.
API Endpoints	GET /api/v1/track?orderNumber=&phone=
Done When	Guest tracking works without login but requires phone match.

11.4 Returns & RTO handling

Description	Return request approved by admin generates a reverse pickup (or customer ships back), inspection step, then refund + restock decision. RTO (return to origin) shipments restock and refund based on policy.
Database	return_requests: id, order_id, status (REQUESTED/APPROVED/PICKED_UP/RECEIVED/REJECTED/REFUNDED), reason, photos JSONB, admin_note, created_at. return_items: id, return_id, order_item_id, qty, condition, restock BOOLEAN.
Done When	A return moves from request to refund with stock and credit note handled correctly.

Phase 12: Security, Caching & Performance

Goal: make it fast and hard to abuse - this is a payments site

12.1 Application security hardening

Description	helmet + strict CORS whitelist (customer and admin origins only), body size limits, input sanitisation, parameterised queries only (Prisma), secure cookies, secrets only from env/secret manager, dependency audit (pnpm audit, Dependabot), file upload restrictions, no stack traces, no sensitive data in logs. Go through the OWASP API Top 10 (BOLA/IDOR, broken auth, mass assignment, excessive data exposure, rate limiting, misconfiguration).
Validation & Rules	Response DTOs expose only whitelisted fields (never password_hash, token hashes, internal notes, cost price). Written checklist in /docs/security.md with each item marked and explained.
Done When	Checklist complete; automated IDOR tests pass for orders, addresses, invoices, quotes.

12.2 Security headers & edge protection

Description	Set HSTS, CSP (allow only Razorpay/Stripe/GA/Meta/R2 origins), X-Frame-Options/frame-ancestors, X-Content-Type-Options, Referrer-Policy, Permissions-Policy via Next.js headers + Cloudflare. Cloudflare: proxy on, WAF managed rules, bot fight mode, rate limiting rules, always-HTTPS, minimum TLS 1.2. Admin domain behind Cloudflare Access or IP allowlist.
Validation & Rules	Test with securityheaders.com and SSL Labs; fix CSP violations in report-only mode first, then enforce.
Done When	Security headers grade A; admin domain not reachable from random IPs (if allowlist chosen).

12.3 Rate limiting & abuse protection

Description	Global limiter plus stricter limits on login, register, OTP send/verify, forgot-password, checkout, payment creation, quote requests and contact form. Redis or Postgres-backed sliding window per IP and per user. Add CAPTCHA (reCAPTCHA v3 / Cloudflare Turnstile) on register, login after failures, contact and OTP send.
Validation & Rules	Return 429 with Retry-After. Different limits per route group. Whitelist webhook IPs/paths from CAPTCHA.
Done When	Abusive client gets 429; normal users unaffected.

12.4 Caching

Description	Cache-aside for categories, home sections and product lists with TTL 60-300 s and invalidation on admin updates; Next.js ISR/revalidate for product and category pages; Cloudflare caching for images/static assets. Never cache stock/availability for long (a few seconds at most) and never cache cart, orders or user data publicly.
Validation & Rules	Cache keys include all filters. Protect against stampede (TTL jitter or lock).
Done When	Second request to a listing is served from cache; editing a product refreshes it.

12.5 Database tuning & load test

Description	Run EXPLAIN ANALYZE on listing, search, order list and dashboard queries; add proven composite indexes; remove N+1 queries; tune connection pool. Load test listing, product page, cart and checkout with k6 and record p95 before/after caching.
Validation & Rules	Do not add indexes blindly - prove each with EXPLAIN output. Document numbers in /docs.
Done When	At least two queries tuned with documented before/after timings; checkout p95 recorded.

12.6 Backups & disaster recovery

Description	Daily automated PostgreSQL backups (provider backups + your own nightly pg_dump to R2), R2 versioning for uploads, written restore runbook, and an actual restore drill into a scratch database.
Validation & Rules	Encrypt backups, keep 14-30 days, alert when a backup fails.
Done When	You restored a backup into a fresh DB and the app ran on it.

Phase 13: Testing

Goal: refactor and launch with confidence (write tests along with every phase; use this phase to fill gaps)

13.1 Unit tests

Description	Pricing/GST engine, order state machine, stock service, refund calculations, OTP service, invoice numbering, quote validity, validation schemas.
Done When	Services layer coverage >= 70%; pricing engine ~100% branch coverage.

13.2 Integration tests (API + real DB)

Description	Vitest/Jest + Supertest with a real PostgreSQL test DB via docker-compose; reset between tests; factories for test data. Cover auth, catalog CRUD, cart, checkout, order flow, admin manual order, refunds, RBAC/IDOR.
Validation & Rules	Tests must not depend on execution order.
Done When	Register > login > add to cart > checkout > (mock) pay > invoice flow runs green.

13.3 Payment, webhook & concurrency tests

Description	Mocked Razorpay/Stripe payloads: valid, invalid signature, duplicate, out-of-order, late payment after expiry, captured-but-webhook-missed (reconciliation). Concurrency script: 50 parallel checkouts for last stock unit; overlapping carts; same Idempotency-Key twice.
Done When	All cases pass 20 runs in a row (not flaky).

13.4 Frontend & end-to-end tests

Description	Playwright E2E for: browse > product > cart > checkout (test mode), login/register, admin login > create product > manual order > refund. Component tests for forms. Mobile viewport run.
Done When	E2E suite runs headless in CI.

13.5 Security & UAT

Description	Security test pass (headers, rate limits, JWT tampering, IDOR, file upload, CSRF) and UAT: the client and 2-3 real users complete real flows from a checklist; log bugs, fix, retest.
Done When	UAT checklist signed off by the client in writing.

13.6 CI automation

Description	GitHub Actions on every push: install, lint, typecheck, unit + integration tests (Postgres service container), build all apps, fail the build on any error.
Done When	Clean clone + one command runs all checks green.

Phase 14: DevOps, Deployment & Observability

Goal: ship it like a real service with monitoring and a safe release process

14.1 Environments & domains

Description	Three environments: local, staging, production. Domains: unicotrader.com + www (customer, Vercel), admin.unicotrader.com (Vercel), api.unicotrader.com (Railway), media.unicotrader.com (R2). Staging on subdomains/preview URLs with test gateway keys. Separate env vars per environment.
Validation & Rules	Never use production keys in staging. Staging is password-protected and noindex.
Done When	Staging mirrors production and is used for client review.

14.2 Dockerize API & worker

Description	Multi-stage Dockerfile (small image, non-root user) for API and worker, docker-compose for local (api, worker, postgres, optional redis). Health checks and DB volume.
Validation & Rules	No secrets in image or Git. Use .dockerignore.
Done When	docker compose up starts the full backend locally.

14.3 CI/CD pipeline

Description	GitHub Actions: lint > test > build > deploy. Frontends auto-deploy via Vercel; API/worker deploy to Railway/Render. Database migrations run as a controlled deploy step (prisma migrate deploy) before the new API starts. Deploy to production only from main after all checks pass; keep a rollback procedure.
Done When	Merge to main updates staging automatically; production deploy needs a manual approval click.

14.4 Health, readiness & observability

Description	GET /health (process alive) and GET /ready (DB reachable). Sentry for frontend + backend errors, UptimeRobot checks on home, API health and checkout page with phone/email alerts, protected /metrics (request count, latency, error rate, queue length) if you add Prometheus/Grafana, payment success-rate dashboard.
Validation & Rules	Do not expose /metrics publicly.
Done When	You can answer 'which endpoint is slowest right now and how many payments failed today?' from data.

14.5 Production cutover prep

Description	Switch Razorpay/Stripe from test to live keys and live webhook URLs, set live Brevo/MSG91/WATI credentials, verify DKIM/SPF/DMARC, add Merchant Center and Search Console, configure Cloudflare caching and rules, load production seed (settings, categories).
Done When	Production checklist in /docs/launch.md is 100% ticked.

Phase 15: Data Migration, SEO Migration & Go-Live

Goal: move from the old WordPress site to the new store with zero ranking loss and zero panic

15.1 Content & data migration

Description	Export products/categories from the old WooCommerce/WordPress (CSV/WP export), clean in Excel, map to your import template, import into staging with the bulk importer, attach images, fix SKUs/HSN/GST rates, review every category manually. Migrate existing customers only if the client has consent; otherwise let them register fresh.
Validation & Rules	Keep a spreadsheet of old URL > new URL for every product/category/page.
Done When	Staging contains the full catalogue and the client has spot-checked 50 products.

15.2 SEO migration (301 redirects)

Description	Crawl the old site (Screaming Frog free / sitemap) to list indexed URLs, create a redirect map old > new, implement 301s in Next.js/Cloudflare, keep titles/meta close to the old ones for ranking pages, submit new sitemap, monitor Search Console for 404s for 4 weeks.
Done When	Top 100 old URLs all return 301 to the correct new page on staging.

15.3 Dress rehearsal on staging

Description	Run a full day as if live: staff create offline orders, you place online test-mode orders, process refunds, cancel an order, handle a review-queue order, export GST report, restore a backup. Fix everything found.
Done When	Zero blockers on the rehearsal checklist.

15.4 Staff training & documentation

Description	Write a short admin manual (screenshots + steps for manual order, stock update, shipping, refund, reports) and record a 15-20 minute screen-share video. Train 1-2 staff and the owner. Provide a 'what to do when...' sheet (payment stuck, customer says money cut, site down).
Done When	Staff can create an offline order and process a shipment without your help.

15.5 DNS cutover & go-live

Description	Choose a low-traffic window. Lower DNS TTL 24h before. Point domain to Cloudflare > Vercel/Railway, enable the 301 redirects, verify SSL, run smoke tests (browse, register, a real Rs. 1 live transaction and its refund, invoice, email/SMS/WhatsApp), watch Sentry and logs live.
Validation & Rules	Keep the old site available (renamed subdomain) for 2-4 weeks as a fallback. Have the rollback plan written before you start.
Done When	Real Rs. 1 payment succeeds end to end and is refunded; site is live over HTTPS.

15.6 Post-launch monitoring (first 2 weeks)

Description	Daily: errors in Sentry, failed payments, reconciliation result, Search Console crawl errors, slow pages. Collect feedback from staff and customers, fix quick wins, write a short post-launch report and decide the Phase 16 priorities with the client.
Done When	Two clean weeks with no payment inconsistencies; Phase 16 priority list agreed.

Phase 16: Growth Features (After Launch)

Goal: conversion, retention and SEO features, built in the order of business impact

16.1 Reviews, ratings & Q&A

Description	Verified-buyer reviews after delivery (email request), star rating, photos, admin moderation, product Q&A answered by admin. Add Review/AggregateRating schema.
Database	reviews: id, product_id, user_id, order_item_id UNIQUE, rating 1-5, title, body, status (PENDING/APPROVED/REJECTED), created_at.
Done When	Only buyers of the item can review; rating shows on listing and in Google.

16.2 Coupons & discounts

Description	Percent/flat coupons with validity window, minimum amount, per-user and global usage limits, category/product scope. Usage counter must stay correct under parallel use.
Validation & Rules	Atomic increment with condition. Discount never makes total negative. Applied through the pricing engine only.
Database	coupons: id, code UNIQUE, type, value, min_amount_paise, max_uses, used_count, per_user_limit, valid_from, valid_until, scope JSONB, is_active.
Done When	100 parallel uses of a 10-use code give exactly 10 successes.

16.3 Wishlist, compare & recently viewed

Description	Wishlist for logged-in users, compare up to 4 products by spec table, recently viewed strip.
Done When	Compare table highlights differing specs.

16.4 Abandoned cart & marketing automation

Description	Email/WhatsApp reminders at 1h and 24h for carts with items (consented users only), back-in-stock notifications, post-purchase review and reorder reminders. Unsubscribe handling.
Done When	Reminder sent once per cart and stops after purchase.

16.5 CMS, blog & SEO content

Description	Admin-managed banners, homepage sections and a blog (markdown/rich text) for buying guides and comparisons (how to choose an energy meter, MCB vs MCCB...). Internal linking to products.
Done When	Non-technical staff can publish a blog post.

16.6 Support tickets & live chat

Description	Ticket system linked to orders, status and replies by email, plus Tawk.to or WhatsApp click-to-chat widget.
Done When	Customer raises a ticket from an order and gets an email reply.

16.7 Full RBAC & granular permissions

Description	Permission-based roles (orders.view, orders.refund, stock.adjust, reports.view, settings.edit...) with custom roles created by admin.
Done When	A custom 'Accounts' role can see GST reports but cannot edit products.

16.8 Google Merchant Center & ads readiness

Description	Product feed (XML/Content API), conversion tracking, remarketing audiences, GA4 e-commerce events (view_item, add_to_cart, begin_checkout, purchase).
Done When	Products appear in Merchant Center without errors; purchase events match real orders.

16.9 Multi-language, social login & loyalty (optional)

Description	Hindi/Gujarati UI, Google login (OAuth with state/PKCE, link only verified emails), referral/loyalty points, EMI calculator on product pages, 360/video media.
Done When	Pick only what the data from the first 2-3 months says is worth it.

Phase 17: Senior-Level Bonus (Optional)

Goal: engineering-quality extras that make the project portfolio-grade

17.1 Outbox pattern & event-driven side effects

Description	Write domain events (order confirmed, payment captured) to an outbox table in the same transaction and publish them reliably afterwards to the notification, invoice and analytics workers.
Database	outbox_events: id BIGSERIAL, aggregate_type, aggregate_id, event_type, payload JSONB, published_at NULL, created_at.
Done When	No event is lost even if the app crashes right after commit.

17.2 Realtime admin updates

Description	Server-Sent Events/WebSocket to refresh admin dashboard, new-order notification sound and live stock changes.
Done When	Two admins see a new order appear without refreshing.

17.3 Architecture write-up

Description	README with architecture diagram, ER diagram, order + payment sequence diagram (checkout > pay > webhook > confirm > invoice), ADRs (why custom stack, why pg-boss, why Razorpay), load test results and a 'what I would do at 100x traffic' section.
Done When	A stranger can understand and run the project from the README alone.

Edge cases to test

These are the scenarios that separate a working demo from a reliable store. Write an automated test for each one.

- Two customers try to buy the last unit at the same time.
- Customer pays, bank debits money, but the gateway shows pending/failed.
- Customer double-clicks Pay or pays twice for one order.
- Customer closes the browser in the middle of payment.
- Razorpay webhook is delayed, duplicated, delivered out of order or never arrives.
- Payment is captured after the order already expired or stock was sold out.
- Gateway or your API is down at the moment of payment.
- Same Idempotency-Key arrives in two parallel checkout requests.
- Admin changes a product price while it is sitting in someone's cart.
- Admin enters an offline sale and an online order hits the same stock at the same time.
- High-value order is rejected after payment: refund must happen and stock must be released.
- Partial refund for one item of a multi-item order (credit note + payment status).
- Two invoices generated at the same instant must get different consecutive numbers.
- Intra-state vs inter-state vs international GST calculation, including rounding.
- International order shows the 3.5% fee at checkout and on the invoice, and the amount charged matches.
- Customer A tries to open Customer B's order, invoice, address or quote (IDOR).
- Staff tries to open admin-only pages or API routes.
- Refresh token used twice (theft detection).
- Bulk import file with duplicate SKUs, wrong GST rates, missing images.
- Worker crashes in the middle of an email/invoice job (job must not be lost or duplicated).
- Courier webhook reports delivered for an order that is still PENDING_REVIEW.
- Redis/queue provider is down - does checkout still work and recover later?
- Database backup restore: can you actually bring the system back?

Suggested timeline

Assumes a solo developer working focused hours, about 14 weeks to go-live plus post-launch work. Add buffer for client feedback and KYC/approval delays; adjust around your regular job and freelance work.

When	What
Week 1	Phase 0: decisions, monorepo, ER diagram, design system start, accounts and KYC started
Weeks 2-3	Phase 1: authentication, OTP, RBAC. Design: wireframes finished
Weeks 3-4	Phase 2 + Phase 3: catalog, media, import, stock service
Weeks 5-6	Phase 4: cart, pricing/GST engine, orders, checkout, quotes
Weeks 6-7	Phase 5: payments, webhooks, reconciliation, refunds, international
Week 7	Phase 6 + Phase 7: fraud controls, GST invoices, policy pages
Weeks 8-9	Phase 8: customer portal (can start from Week 5 against mocked APIs)
Weeks 9-11	Phase 9 + Phase 10 + Phase 11: admin portal, notifications, shipping
Week 12	Phase 12: security, caching, performance, backups
Week 13	Phase 13: test gaps, E2E, UAT
Week 14	Phase 14 + Phase 15: deploy, data/SEO migration, rehearsal, go-live
Weeks 15-18	Post-launch monitoring, then Phase 16 growth features in business-impact order

Go-live checklist

- Razorpay live keys + live webhook URL set; webhook signature verified in production.
- International gateway live (or international selling disabled with a clear message).
- GST number, registered address, invoice prefix and starting number configured and approved by the CA.
- All policy pages live; T&C consent required at checkout; grievance officer details shown.
- SPF, DKIM and DMARC pass; emails land in inbox, not spam.
- SMS DLT templates approved; WhatsApp templates approved.
- Security headers grade A; Cloudflare WAF and rate limiting on; admin has 2FA.
- Backups running and one restore drill completed.
- Sentry and UptimeRobot alerts reach your phone.
- 301 redirect map tested; sitemap submitted; Search Console and Merchant Center connected.
- Rs. 1 live payment + refund done; invoice and notifications verified.
- Staff trained; admin manual and 'what to do when...' sheet delivered.
- Old site kept as fallback; rollback plan written down.

Note: cost, GST and legal points here are planning guidance, not legal or tax advice. Confirm them with a qualified CA/lawyer before launch.